LAPORAN TUGAS BESAR II

IMPLEMENTASI MEKANISME *CONCURRENCY CONTROL* DAN *RECOVERY*

IF3140 MANAJEMEN BASIS DATA

Dibuat oleh:

13518102	Mohamad Falah Sutawindaya
13518109	Andjani Kiranadewi
13518140	Hansel Grady Daniel Thamrin
13518141	Muhammad Daffa Dinaya
13518146	Naufal Prima Yoriko

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2020

BAB I
PEMBAHASAN

1. Implementasi *Concurrency Control*

Terdapat beberapa algoritma di dalam concurrency control, berikut ini adalah ketiga algoritma yang telah kami implementasikan

1. *Simple Locking (Exclusive locks only)*

   …

   Pada algoritma *Simple Locking*, terdapat dua fase:

   1. *Growing Phase* : Saat transaksi pertama kali memasuki sistem, transaksi tersebut akan memperoleh *lock* untuk semua *item-item* yang akan di *read* ataupun *write*.

      Algoritma *Simple Locking* pada laporan ini hanya mengimplementasi *Exclusive Lock* dimana:

      - Apabila sebuah item disaat permintaan *lock* tidak dimiliki transaksi apapun, maka transaksi yang memintanya akan mendapatkan *lock* pada item tersebut
      - Apabila sebuah item disaat permintaan *lock* dimiliki oleh sebuah transaksi, maka akan dilakukan salah satu dari langkah berikut:
        - Jika transaksi tersebut hanya memiliki satu operasi (baik *write* atau *read*), maka transaksi tersebut akan memasuki status *wait*.
        - Jika transaksi tersebut memiliki lebih dari satu operasi, maka transaksi tersebut akan di-*abort* dan akan dimasukkan kembali ke dalam antrian transaksi.
   2. *Shrinking Phase* : Fase ini terjadi setelah transaksi di-*abort* atau menyelesaikan semua operasi yang dilakukannya. Dilakukan *release* atau *unlock* pada semua *lock* yang telah diperoleh transaksi tersebut.

   Berikut pseudocode nya

   a. *RunLockingScheduler*

      Prosedur ini akan menjalankan transaksi berdasarkan algoritma *Simple Locking*. Transaksi yang masuk akan diiterasi *read_set* dan *write_set*nya untuk memperoleh *lock* yang diperlukan. Apabila terdapat *lock request* yang gagal dan transaksi melakukan *read* dan *write* pada lebih dari satu *item*, maka transaksi tersebut akan di-*abort* dan dimasukkan kembali kedalam antrian transaksi.

Setelah semua *lock* didapatkan, transaksi dapat melakukan operasi yang diperlukannya sebelum akhirnya semua *lock* di-*release* kembali.

```
procedure RunLockingScheduler(){
        while thread pool is active {
                if txn_requests not empty {
                        txn ← txn_requests.pop()
                        forEach readReq in txn.readset_ do {
                                // if lock request is
                                // rejected
                                if not ReadLock(txn,readReq){
                                        Release all readset
                                        locks

                                        break
                                }
                        }
                        forEach writeReq in txn.writeset_ do
                        {
                                if not
                                WriteLock(txn,readReq){
                                        Release all writeset
                                        locks

                                        break
                                }
                        }
                        if all lock acquired {
                                ready_txn.push_back(txn)
                        } else {
                                // Push transaction back to
                                // txn_request list
                                txn_requests.push_back(txn)
                        }
                }
                // Process all finished transactions
                forEach txn in completed_txn {
                        if txn.status = COMPLETE_C {
                                Commit(txn)
                        }
                        Release all locks
                }
                // Start executing transactions
                forEach txn in ready_txn {
                        runTask(txn)
                }
        }
}
```

b. *ReadLock* & *WriteLock*

Karena algoritma *Simple Locking* ini hanya mengimplementasi *Exclusive Locking*, maka algoritma untuk *request ReadLock* dan *WriteLock* sama.

Pertama, akan dicek terlebih dahulu apakah suatu *item* telah dimiliki *lock*nya oleh transaksi lain. Apabila tidak dimiliki oleh transaksi apapun, maka transaksi yang me*request lock* akan langsung memiliki *lock* tersebut.

```
function ReadLock (txn : Transaction, key: Key): boolean{
        dq ← getLockQueue(key)
        rq ← LockRequest(EXCLUSIVE, txn)

        empty = dq->empty()
        dq->push_back(rq)

        // If other transaction is holding the lock
        if (not empty){
                // Add transaction to wait list
                txn_waits_[txn]++
        }
        → empty

}
```

c. *Release*

Prosedur ini akan melakukan *release* atau *unlock* terhadap *lock* ataupun *lock request* dari sebuah transaksi pada suatu item. Pertama, daftar *lock queue* pada suatu item akan diperoleh. Kemudian, *lock request* dari transaksi yang memanggil prosedur ini akan dihapus dari *queue* tersebut. Apabila transaksi tersebut merupakan pemilik *lock item* tersebut, maka transaksi selanjutnya di dalam *queue* tersebut akan memperoleh *lock item*.

```
procedure Release (txn : Transaction, key: Key){
        locks ← getLockQueue(key)
        isOwner ← true

        forEach lock in locks {
                if lock.txn = txn {
                        locks.erase(lock)
                        break
                }
                isOwner ← false
        }
        // If the released lock is in front of the queue
        if (not locks.empty and isOwner){
                Next in queue gets the lock
        }

}
```

```
                              Average Transaction Duration
                0.1ms           1ms             10ms
'Low contention' Read only (5 records)
 Locking A      44001.7         4816.72         505.282
'Low contention' Read only (20 records)
 Locking A      22823.5         4886.99         494.224
'High contention' Read only (5 records)
 Locking A      31987.2         3963.77         438.303
'High contention' Read only (20 records)
 Locking A      5771.21         1235.25         130.086
Low contention read-write (5 records)
 Locking A      38132.1         4813.78         478.782
Low contention read-write (10 records)
 Locking A      33084.4         4516.42         495.51
High contention read-write (1 records)
 Locking A      24229.2         3155.6          323.404
High contention read-write (5 records)
 Locking A      31012.8         3953.24         412.037
High contention read-write (10 records)
 Locking A      15243           2762.89         297.542
High contention mixed read only/read-write
 Locking A      4190.66         1060.71         122.467
```

2. *Serial Optimistic Concurrency Control (OCC)*

   Pada algoritma *Serial Optimistic Concurrency Control,* terdapat tiga fase utama dalam menjalankan algoritma. Fase-fase tersebut adalah sebagai berikut

   1. *Read and execution phase* : Pada fase ini, awalnya transaksi akan diberi timestamp saat memulai transaksi. Kemudian transaksi akan ditulis ke dalam temporary variable yang disini nanti akan ditulis dalam set.
   2. *Validation phase* : Pada fase ini akan dilakukan validasi terhadap serializability pada transaksi yang dilakukan. Syaratnya adalah sebagai berikut
      a. Untuk semua transaksi dengan key *i* dan transaksi dengan key *j* dan $TS(T_i) < TS(T_j)$ maka memenuhi
         i. $finishTS(T_i) < finishTS(T_j)$: eksekusi transaksi tidak konkuren
         ii. $startTS(T_j) < finishTS(T_i) < validationTS(T_j)$ dan data yang ditulis oleh T_i tidak beririsan dengan data yang dibaca T_j : Eksekusi secara konkuren namun T_j tidak membaca data yang ditulis T_i
      b. Atau validasi gagal dan T_j *aborted*
   3. *Write phase* : Jika transaksi valid maka akan di-*commit* ke database jika tidak maka akan di-*rollback*

Berikut pseudocode nya

a. *RunOCCScheduler*

Pada prosedur ini, program akan mulai menjalankan transaksi dengan algoritma OCC. Di awal prosedur, semua transaksi yang ada di dalam antrian transaksi yang di-*request* akan dijalankan terlebih dahulu. Kemudian ketika transaksi yang ada di list request telah selesai maka akan diberi status sesuai dengan *case* nya masing-masing

1. Meng-assign task transaksi ke thread pool
2. Kemudian pada setiap transaksi akan dilakukan sebagai berikut
3. Jika masih terdapat transaksi yang perlu diproses di *list*, maka diproses
4. Jika terdapat transaksi yang sudah selesai diproses maka status akan ditentukan berdasar sinyal status yang diberikan ketika transaksi sudah diproses
    a. Jika COMPLETED_ABORT maka akan diberi status abort
    b. Jika COMPLETED_COMMIT maka dan valid maka akan diberi status commit
    c. Jika tidak valid maka transaksi akan dibersihkan dan di-*redo*

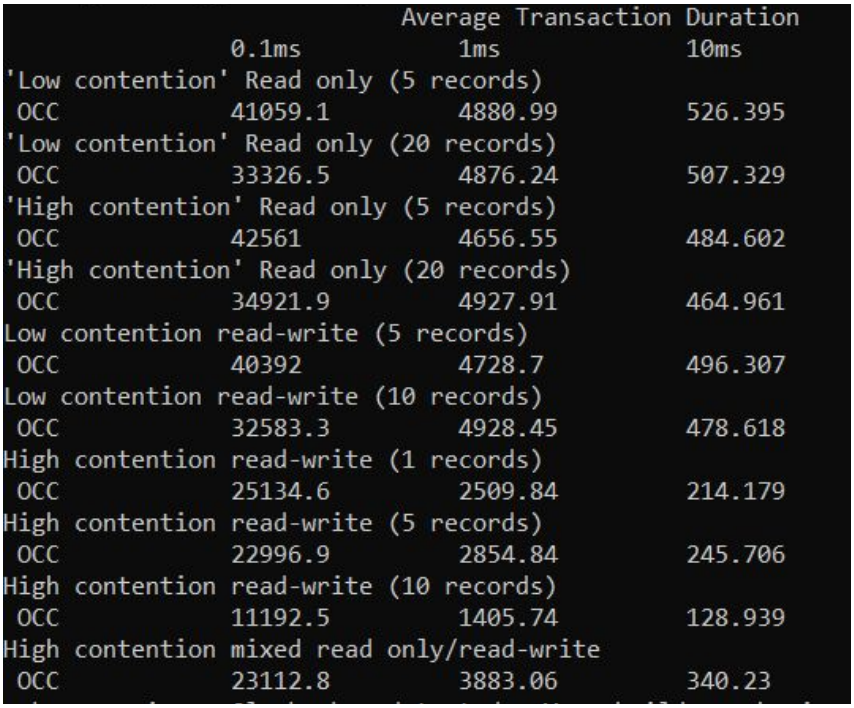
```
procedure RunOCCScheduler(output txn : Transaction){
      while thread_pool is still active do
            if txn_request_list is not empty then
                  txn ← txn_request_list.pop()
                  txn.run()
            if txn_finished_list is not empty then
                  txn ← txn_finished_list.pop()
                  if txn.status is COMPLETED_ABORT_SIGNAL
                  then
                        txn.status ← ABORTED
                  else
                        validate txn
                        if valid then
                              COMMIT()
                        else
                              CLEAN UP()
                              RESTART()
}
```

b. *OCCValidateTransaction*

Pada fungsi ini, program akan melakukan pengecekan apakah sebuah transaksi valid atau tidak. Pada setiap *set* yang perlu dibaca untuk melakukan transaksi dan *set* yang mungkin akan ter-*update* setelah transaksi akan diperiksa *timestamp*-nya. Jika timestamp pada storage lebih baru daripada waktu dimulainya transaksi maka transaksi tidak valid

```
function OCCValidateTransaction(txn : Transaction):
boolean{
      foreach key in txn.readset_ do
            if txn.start_time < storage.timestamp with
appropriate key
            then
                → false
      foreach key in txn.writeset_ do
            if txn.start_time < storage.timestamp with
appropriate key
            then
                → false
}
```

Dari algoritma yang kami implementasi, kami memiliki hasil percobaan tunggal (belum dibandingkan dengan algoritma lain), yaitu seperti pada gambar berikut ini



```
                              Average Transaction Duration
                0.1ms            1ms              10ms
'Low contention' Read only (5 records)
 OCC            41059.1          4880.99          526.395
'Low contention' Read only (20 records)
 OCC            33326.5          4876.24          507.329
'High contention' Read only (5 records)
 OCC            42561            4656.55          484.602
'High contention' Read only (20 records)
 OCC            34921.9          4927.91          464.961
Low contention read-write (5 records)
 OCC            40392            4728.7           496.307
Low contention read-write (10 records)
 OCC            32583.3          4928.45          478.618
High contention read-write (1 records)
 OCC            25134.6          2509.84          214.179
High contention read-write (5 records)
 OCC            22996.9          2854.84          245.706
High contention read-write (10 records)
 OCC            11192.5          1405.74          128.939
High contention mixed read only/read-write
 OCC            23112.8          3883.06          340.23
```

3. *Multiversion Timestamp Ordering Concurrency Control (MVCC)*

Algoritma ini menggunakan ruang penyimpanan yang tidak tunggal untuk tiap data, melainkan memiliki versi yang bergantung pada operasi *write* pada transaksi, dan *ordering*/urutan didasarkan pada timestamp (di program timestamp diwakilkan dengan *transaction_id*, *transaction_id* yang lebih kecil berarti diasumsikan lebih awal).

Jika transaksi yang melakukan operasi dinyatakan dalam T_i dan Obj_k adalah versi paling recent (versinya (atau *write timestamp*) lebih kecil atau sama

dengan timestamp transaksi T_i) dari Obj, maka berikut ini adalah langkah dari algoritma ini (didasarkan pada slide kuliah):

a. Read

Mengembalikan value dari Obj_k, jika read timestamp dari Obj_k lebih dari timestamp dari T_i, maka *read timestamp*-nya diupdate dengan timestamp dari T_i.

Berikut ini adalah *pseudocode*-nya

```
/* Disini Ti adalah txn, Obj adalah key, hasil dari data
yang ingin dituliskan adalah result, dan Objk adalah V */

function read(input txn: Transaction, key: Key, output
result: Value): boolean {
    /* Mengecek apakah ada di database */
    if existKey(key) then
        V :  Version

        /* Mendapatkan versi terbaru yang tidak lebih besar
dari txn.timestamp */
        v ← getRecentVersion(txn, key)
        result ← v.value

        if(v.readTimestamp < txn.timestamp) then
            v.readTimestamp ← txn.timestamp

        → true

    → false
}
```

b. Write

Untuk algoritma ini dapat dibagi menjadi beberapa *case*:

1. Jika timestamp dari T_i lebih kecil daripada *read timestamp* dari Obj_k, maka transaksi T_i akan di-*rollback* dan tidak ada perubahan terjadi pada Obj_k.
2. Jika timestamp dari T_i sama dengan *write timestamp* dari Obj_k, maka *value* dari Obj_k akan di-*overwrite* dengan *value* yang baru dari transaksi.
3. Pada kasus selain itu (*timestamp* dari T_i lebih besar daripada *write timestamp* dari Obj_k), maka akan dibuat versi baru dari Obj dengan *read timestamp* dan *write timestamp* yang sesuai dengan *timestamp* dari T_i dan *value*-nya sesuai dengan *value* yang baru diberikan di fungsi ini.

Pada implementasi, fungsi *write* akan dibagi menjadi dua fungsi berbeda yaitu

1. CheckWrite, mengecek apakah operasi *write* mungkin dilakukan
2. Write, melakukan penulisan versi baru dari Obj setelah dipastikan bahwa penulisan dapat dilakukan dengan fungsi CheckWrite.

Berikut ini adalah *pseudocode*-nya

```
/* Disini Ti adalah txn, Obj adalah key, dan Objk adalah V
*/
function checkWrite(input txn: Transaction, input key:
Key): boolean {
    /* Mengecek apakah ada di database */
    if existKey(key)then
        V : version
        /* Mendapatkan versi terbaru yang tidak lebih besar
dari txn.timestamp */
        v ← getRecentVersion(txn, key)

        if v.readTimestamp <= txn.timestamp then
            return true

        → false

    → false
}

/* Disini Ti adalah txn, Obj adalah key, nilai dari data
yang ingin dituliskan adalah newValue, dan Objk adalah V */
procedure write(input txn: Transaction, input key: Key,
input newValue: Value) {
    /* Mengecek apakah ada di database */
    if existKey(key) then
        V : version

        /* Mendapatkan versi terbaru yang tidak lebih besar
dari txn.timestamp */
        v ←  getRecentVersion(txn, key)

        if v.writeTimestamp == txn.timestamp then
            v.value ←  newValue

    /* Menambah entri data baru di database
        dimana readTimestamp dan writeTimestamp =
txn.timestamp */
    addEntry(key, newValue)
}
```

Selain itu MVCC memiliki scheduler pada program yang berfungsi untuk menyimulasikan transaksi-transaksi yang berjalan konkuren, dan kemudian di *schedule* jadwal *read/write* tiap transaksi oleh *scheduler* ini dengan menggunakan *thread* yang berbeda (*multi-threading*), yang fungsi

read dan writenya menggunakan algoritma yang sama seperti yang dituliskan diatas. Cara kerja MVCC *scheduler* ini kurang lebih

1. Meng-assign tiap transaksi yang ingin dilakukan ke dalam *thread* yang berbeda untuk dijalankan secara konkuren.
2. Pada tiap thread,
    - 2.1. *Read* tiap entri data yang dibutuhkan, baik yang akan di-*read* maupun di-*write*. Sebelum *read* data, data terkait akan di-*lock*, dan *lock* dilepaskan setelah selesai dibaca (menghindari modifikasi saat proses pembacaan).
    - 2.2. Menjalankan *logic* dari transaksi (yang tidak terkait langsung dengan *database*, namun biasanya terkait dengan operasi di *buffer* transaksi).
    - 2.3. Melakukan *locking* ke seluruh data yang akan dituliskan.
    - 2.4. Melakukan pengecekan apakah setiap *key* mungkin dilakukan penulisan.
    - 2.5. Apabila ada satu *key* gagal, maka *locking* dilepas, kemudian transaksi di reset dan dimasukkan ulang ke antrian eksekusi transaksi.
    - 2.6. Apabila seluruhnya berhasil, maka seluruh operasi *write* dijalankan dan kemudian status diubah menjadi *committed.*

Berikut ini adalah *pseudocode*-nya

```
procedure MVCCScheduler() {
    while notEmpty(txnRequest) do
        txn: Transaction

        txn ← pop(txnRequest)

        /* menjalankan tiap transaksi di thread berbeda */
        runMVCCTxn(txn)
    }
}

procedure runMVCCTxn(input txn: Transaction) {
    foreach readReq in readSet do
        lockData(readReq.key)
        read(txn, readReq.key, readReq.value)
        unlockData(readReq.key)
    }

    foreach writeReq in writeSet do
        lockData(writeReq.key)
        read(txn, writeReq.key, writeReq.value)
        unlockData(writeReq.key)
    }
```

```
    /* menjalankan logic dari txn */
    run(txn)

    foreach writeReq in writeSet do
        lockData(writeReq.key)
    }

    foreach writeReq in writeSet do
        if not checkWrite(txn, writeReq.key) then
            resetTxn(txn)
            addTxnRequest(txn)
            →

    foreach writeReq in writeSet do
        write(txn, key, writeReq.value)

    foreach writeReq in writeSet do
        unlockData(writeReq.key)

    txn.status ← COMMITTED
}
```

Dari algoritma yang kami implementasi, kami memiliki hasil percobaan tunggal (belum dibandingkan dengan algoritma lain), yaitu seperti pada gambar berikut ini

	Average Transaction Duration		
	0.1ms	1ms	10ms
'Low contention' Read only (5 records)			
MVCC	27339.2	4189.77	424.531
'Low contention' Read only (20 records)			
MVCC	15901.1	4091.65	456.034
'High contention' Read only (5 records)			
MVCC	31003.7	3984.96	505.77
'High contention' Read only (20 records)			
MVCC	18006.5	4049.33	489.336
Low contention read-write (5 records)			
MVCC	26765.5	3369.16	468.81
Low contention read-write (10 records)			
MVCC	17871.6	4025.75	494.229
High contention read-write (1 records)			
MVCC	4172.26	1085.1	134.959
High contention read-write (5 records)			
MVCC	2867.09	1098.13	115.772
High contention read-write (10 records)			
MVCC	1410.58	561.97	72.9296
High contention mixed read only/read-write			
MVCC	4513.78	3409.42	657.26

```
                                Average Transaction Duration
                0.1ms           1ms             10ms
'Low contention' Read only (5 records)
 MVCC           24510.9         3898.35         474.473
'Low contention' Read only (20 records)
 MVCC           15020.1         3418.62         457.825
'High contention' Read only (5 records)
 MVCC           19183.5         3551.16         451.179
'High contention' Read only (20 records)
 MVCC           19195           3665.91         451.798
Low contention read-write (5 records)
 MVCC           22538.1         3783.39         480.411
Low contention read-write (10 records)
 MVCC           11681.9         3267.23         471.493
High contention read-write (1 records)
 MVCC           4326.41         1161.83         129.032
High contention read-write (5 records)
 MVCC           2836.1          865.845         126.119
High contention read-write (10 records)
 MVCC           1240.5          501.618         64.0118
High contention mixed read only/read-write
 MVCC           3671.63         2572.17         609.745
```

4. Hasil analisis dari algoritma yang diterapkan

Untuk tiap algoritma berikut, kami akan melakukan analisis performa algoritma berdasarkan teori, implementasi, ataupun eksekusi dari waktu *real* saat program dijalankan.

a. *Simple Locking Exclusive Locking Only*

i. *Simple locking*, dalam implementasinya, menyimpan *lock table* berisi lock yang sudah diberikan ataupun yang sedang *waiting*. *Lock table* memiliki struktur data yang relatif sederhana, sehingga tidak memakan memori begitu banyak.

ii. Performansi *Simple locking* untuk transaksi *read-only* dengan *low contention* lebih rendah dibanding OCC. Hal ini karena, dibandingkan dengan OCC yang tidak menganggap ada konflik pada transaksi yang dijalankan, *Simple Locking* selalu melakukan locking terhadap semua operasi yang dilakukan baik operasi *read* ataupun *write*, sehingga waktu yang dibutuhkan lebih lama.

iii. Jika dibandingkan dengan OCC, *Simple locking* memiliki performansi yang lebih unggul cukup signifikan di beberapa bagian pengujian transaksi yang memiliki konflik. Pada suatu kasus bahkan hampir 50% dari waktu yang dibutuhkan OCC. Hal

ini karena simple locking memang secara teori lebih unggul untuk menangani transaksi yang terdapat konflik. Terutama ketika kasus *mixed read only/read-write high contention* (berpeluang konflik yang tinggi)

b. Optimistic Concurrency Control
   i. Pada OCC secara teori akan memiliki performansi yang lebih baik ketika terdapat konflik yang sedikit karena OCC tidak akan rollback dan melakukan locking saat rollback.
   ii. Pada hasil uji coba memang benar menunjukan bahwa terdapat saat dimana OCC menunjukan performansi lebih baik saat chance konflik dari transaksi sedikit(*Low contention*), namun ketika waktu *waiting time* dari *thread* diperlama, maka situasi akan berbalik dimana performansi lebih baik untuk *locking*. Hal ini disebabkan cost time untuk mengulangi transaksi lebih mahal karena waktu waiting time dari thread lebih lama dari waktu *lock* dan *release* pada *Simple Locking*.
   iii. Pada operasi yang terdapat write kemungkinan akan semakin tinggi dalam menyebabkan konflik sehingga secara teori akan memperlambat waktu pengerjaan OCC.
   iv. Pada hasil uji memang benar operasi yang melibatkan *write* memiliki waktu pemrosesan yang lebih lama dari simple locking.

c. Multiversion Timestamp (MVCC)
   i. MVCC dapat menjamin bahwa operasi *read* selalu berhasil, dan dengan kata lain transaksi *read-only* akan selalu sukses pula.
   ii. Secara teori, algoritma dari protokol ini akan menjamin bahwa transaksi yang konkuren dapat *serializable*.
   iii. Secara teori maupun implementasi, *multiversion* akan menyimpan versi baru dari data setiap dilakukannya *write* (pada implementasinya ke dalam sebuat deque). Maka, *tradeback* dari implementasi MVCC ini adalah tentu lebih boros memori, terutama jika terdapat operasi *write*. Hal ini lebih terbukti saat operasi dimana kami beberapa kali mendapatkan error 137 saat melakukan eksekusi program yang menandakan r*un-out of memory*.
   iv. Dari implementasi, MVCC memerlukan struktur data yang lebih rumit, sehingga untuk transaksi yang sederhana terkadang struktur yang dapat dibilang '*too powerful*' (dapat terlihat dari implementasi yang memiliki *storage* khusus dibanding algoritma

lain), sehingga akan memiliki *overhead* yang tergolong besar dalam melakukan transaksi tersebut (umumnya yang tergolong sederhana adalah transaksi *read-only*).

v. MVCC memiliki versi untuk setiap *write*, yang artinya walaupun operasi dengan *less prior* (*timestamp* yang besar) memiliki eksekusi *write* yang kebetulan lebih awal, akan tetap aman dan dapat ditulis secara konkuren, artinya pada operasi normal (ada *read* maupun *write*) dapat lebih optimal karena tingkat konkurensi lebih baik.

vi. Dari eksperimen terlihat bahwa secara umum, MVCC memiliki performa waktu yang lebih baik pada operasi dengan contention yang tinggi/high, karena dari strukturnya MVCC cukup tangguh terhadap konflik yang diakibatkan *editing* data yang sama oleh operasi yang berbeda (karena adanya *versioning*), sehingga *rollback* dan *redo* dari suatu transaksi yang *less prior* dapat dikurangi dan meningkatkan performa.

```
'Low contention' Read only (5 records)
 MVCC           21253.8         3992.27         446.316
 OCC            29154.9         4031.55         496.699
 Locking A      25805           4120.42         485.649
'Low contention' Read only (20 records)
 MVCC           20378.1         3805.49         458.4
 OCC            22430.7         3504            445.603
 Locking A      8862.9          3725.25         439.126
'High contention' Read only (5 records)
 MVCC           25329.1         4154.07         455.281
 OCC            31557.5         4252.31         488.161
 Locking A      15524.7         3363.18         405.451
'High contention' Read only (20 records)
 MVCC           17120.3         4348.35         476.784
 OCC            24400.1         4353.09         439.828
 Locking A      3107.3          875.292         120.049
Low contention read-write (5 records)
 MVCC           20713.6         3295.08         393.159
 OCC            26722.6         4133.4          467.066
 Locking A      23051.3         3819.74         490.296
Low contention read-write (10 records)
 MVCC           17310.8         3687.23         445.753
 OCC            15706.9         4012.6          446.806
 Locking A      14887.2         4086.85         500.447
High contention read-write (1 records)
 MVCC           3945.24         1026.15         132.447
 OCC            13057.2         2162.13         217.587
 Locking A      15095.5         2900.72         296.93
High contention read-write (5 records)
 MVCC           3556.91         1012.74         120.321
 OCC            14343.4         1963.75         241.646
 Locking A      12404.8         2814.37         386.01
High contention read-write (10 records)
 MVCC           1234.06         549.999         64.7103
 OCC            5114.11         1130.51         120.564
 Locking A      6568.2          1826.54         275.34
High contention mixed read only/read-write
 MVCC           4153.01         2564.08         538.371
 OCC            9325.92         2646.2          265.46
 Locking A      2359.72         758.073         105.789
```

```
                                Average Transaction Duration
                  0.1ms            1ms              10ms
Low contention read-write (5 records)
 MVCC             15972.6          3427.99          391.973
 OCC              28528.9          4212.86          446.887
 Locking A        20256.1          3827.6           477.363
Low contention read-write (10 records)
 MVCC             17936.5          3542.25          426.374
 OCC              21419.1          3468.54          452.701
 Locking A        12178            3771.78          476.761
High contention read-write (1 records)
 MVCC             3926.72          923.664          122.336
 OCC              14975.9          2493.96          210.124
 Locking A        18439.4          2317.57          274.023
High contention read-write (5 records)
 MVCC             3316.47          988.851          116.067
 OCC              11150.4          1994.12          235.86
 Locking A        10930.1          2828.87          406.172
High contention read-write (10 records)
 MVCC             1191.36          592.059          65.756
 OCC              8510.59          1206.27          120.593
 Locking A        8633.83          2302.88          268.079
High contention mixed read only/read-write
 MVCC             4051.02          3106.64          525.422
 OCC              12482.2          2608.55          273.598
 Locking A        3137.6           755.904          112.982
```

Maka berdasarkan hasil masing-masing diatas, kami membuat komparasi dari algoritma yang kami kerjakan dan juga membuat eksekusi yang membandingkan tiap algoritma secara langsung (hasil eksekusi terdapat pada gambar diatas), dengan tujuan dapat menemukan dan menyimpulkan *use case* yang sesuai untuk tiap algoritma. Berikut ini adalah hasil kesimpulan dari analisis kami

a. Secara rata-rata, algoritma ***simple locking*** **dengan** ***exclusive-only lock*** **terlihat memiliki keunggulan** dibanding algoritma lainnya **pada operasi** ***read only*** **dan memiliki** ***contention yang tinggi*** berdasarkan indikator waktu eksekusi. Secara teoritik dan praktik dari kode, kami menyimpulkan bahwa hal ini diakibatkan metode locking memiliki struktur yang cukup sederhana, sehingga memiliki overhead operasi yang relatif lebih kecil dibandingkan algoritma lain. Selain itu, mekanisme untuk membatasi akses transaksi lain juga cukup efektif sehingga dapat memangkas waktu

operasi selama proses locking terjadi dengan aman (tidak ada *dependency loop* yang mengakibatkan *deadlock)*.

b. Secara rata-rata, pada operasi yang memiliki waiting time lebih kecil, *Simple Locking* memiliki waktu eksekusi yang relatif lebih lambat, sebaliknya MVCC memiliki waktu eksekusi yang lebih cepat. Hal ini kami simpulkan akibat *Locking* bersifat melakukan *blocking* pada eksekusi lain sehingga membatasi derajat konkurensi (derajat konkurensi terkait langsung dengan *waiting time*), sedangkan MVCC memiliki derajat konkurensi yang baik akibat adanya *versioning* (seperti yang dijelaskan sebelumya).
c. Secara rata-rata, algoritma **MVCC terlihat memiliki keunggulan** dibanding algoritma lainnya **pada *operasi read-write* atau memiliki *contention* yang tinggi/*high*** berdasarkan indikator waktu eksekusi. Secara teoritik dan praktik dari kode, kami menyimpulkan bahwa hal ini diakibatkan metode MVCC memiliki *versioning* sistem yang membuatnya tangguh terhadap kemungkinan terjadinya konflik akibat akses data yang sama, dan akan mengecilkan peluang terjadinya *rollback* suatu transaksi dan secara tidak langsung akan meningkatkan derajat konkurensi dari algoritma ini. Selain itu, pada operasi *write* sekalipun memiliki kemungkinan konflik yang kecil semenjak algoritma memiliki opsi untuk membuat versi yang baru.
d. Meninjau dari **sisi penggunaan memori, MVCC memiliki kekurangan dibanding yang lain**, akibat struktur data yang lebih kompleks (lebih banyak blocking di memori) dan juga secara eksplisit akan menyimpan veris berdasarkan *write*-nya, yang artinya akan membuat memori lebih boros dibandingkan simple locking ataupun OCC.
e. Secara teori, **OCC seharusnya memiliki keunggulan pada konkurensi yang memiliki *contention* rendah/*low*** (karena kemungkinan konflik kecil). Hal ini diakibatkan OCC adalah algoritma 'optimis' yang memiliki *base assumption* tidak terjadi konflik pada tiap transaksi, sehingga validasi dapat dilakukan lebih cepat. Namun, sekalinya terjadi konflik sungguhan algoritma ini menjadi kurang efektif (yakni pada *high contention*), sehingga dapat dilihat pada kondisi high contention OCC lebih buruk dari yang lain secara cukup signifikan. Namun secara praktikal hal ini tidak terlalu terbukti, bahkan OCC memiliki performa yang serupa dengan algoritma lainnya (tidak terlihat keunggulan nyata pada OCC).

Dari poin-poin diatas, kami menyimpulkan bahwa

a. Algoritma *simple locking* baik dipakai saat transaksi cukup sederhana (misal hanya mengandung operasi read) dan memiliki *contention* yang cukup tinggi.
b. Algoritma OCC baik dipakai saat *contention* cukup rendah atau konflik (seperti *accessing* data yang sama) cukup rendah, karena memiliki asas dasar 'optimis'.
c. Algoritma MVCC baik dipakai saat transaksi cukup kompleks (melibatkan operasi *read* maupun *write*), tidak berpengaruh terhadap *contention* (namun pada kondisi *contention* tinggi algoritma ini cukup andal), dan sangat baik pada *case* dimana transaksi-transaksi memiliki derajat konkurensi yang cukup tinggi (sangat mendukung *case* ini, dan merupakan salah satu keunggulan utama sistem *versioning*), namun memiliki tradeback harus digunakan pada sistem dengan RAM yang cukup besar, karena memakan banyak *working memory* untuk menyimpan data yang mungkin memiliki banyak versi.

2. Eksplorasi *Recovery*

Pada DBMS PostgreSQl, terdapat 3 mekanisme *recovery* yang digunakan di dalam DBMS tersebut. Berikut ini adalah penjelasan mengenai ketiga mekanisme *recovery* tersebut.

1. *Write-Ahead Log*

   Write-Ahead Log adalah salah satu metode *recovery* yang digunakan dalam PostgreSQL. Konsep utama dari *Write-Ahead Log* adalah DBMS akan melakukan pengubahan pada data yang disimpan jika dan hanya jika perubahan yang dibuat pengguna telah dicatat ke dalam memori. Catatan perubahan ini disebut dengan *log*. Dari sini, jika suatu basis data mengalami suatu kejadian yang menyebabkan terjadinya kegagalan pada basis data, maka DBMS akan merujuk kepada *log* terakhir sebelum basis data mengalami kegagalan sebagai acuan untuk mengubah basis data. Perubahan yang belum diterapkan pada data dapat dibuat ulang dengan catatan *log*. Penggunaan *Write-Ahead Log* dapat membuat proses penulisan data ke dalam memori berkurang karena suatu perubahan dianggap telah dilakukan apabila *log* dari suatu perubahan telah dicatat di dalam memori, sehingga pencatatan ini menjadi tanda terjadinya suatu transaksi. *Log* pada DBMS dibuat secara berurutan sesuai dengan urutan transaksi. Proses penyimpanan *log* ini juga memungkinkan DBMS untuk melakukan proses *backup* pada basis data sesuai dengan *log* yang telah tersimpan. Karena setiap *log* disimpan secara *real-time*, maka DBMS dapat merujuk kepada salah satu *log* pada waktu tertentu

untuk memutar ulang *log* yang berkaitan untuk mengembalikan status basis data ataupun untuk memperbaiki inkonsistensi pada basis data.

2. *Continuous Archiving*

   Continuous Archiving adalah salah satu metode *recovery* yang terdapat pada PostgreSQL. Pada dasarnya, basis dari *Continuous Archiving* adalah perubahan *Write-Ahead Log* pada bagian sebelumnya. Namun, pada *Continuous Archiving*, DBMS akan menggunakan *backup* pada tingkatan *file system* dan *Write-Ahead Log* yang telah dibuat berdasarkan semua transaksi yang telah dilakukan di dalam basis data. Dari segi pelaksanaan, jika dibandingkan dengan *Write-Ahead Log*, *Continuous Archiving* lebih kompleks karena proses *recovery* basis data berlangsung pada level *file system* yang disertai dengan pemutaran ulang *log* dari *Write-Ahead Log* sejak penanda (*checkpoint*) terakhir. Terdapat beberapa kelebihan pada metode ini, yaitu pemutaran ulang memungkinkan DBMS tidak membutuhkan cadangan *file system* yang konsisten sempurna sebagai *backup* untuk dijadikan titik acuan basis data. Selain itu, DBMS tidak harus melakukan pemutaran ulang semua *log* untuk memulihkan basis data, namun DBMS hanya perlu memutar ulang *log* hingga basis data sudah menjadi konsisten. Metode ini tergolong membutuhkan banyak penyimpanan, yaitu penyimpanan dasar yang besar dalam bentuk *log-log* yang tercipta. Metode ini cocok digunakan pada basis data yang dituntut untuk andal pada setiap waktu. Metode ini banyak digunakan untuk basis data yang lebih sensitif terhadap akurasi data dengan kekurangannya adalah membutuhkan lebih banyak storage dalam melakukan proses log backup nya. Metode pemutaran ulang log dinamakan sebagai *Point-in-Time Recovery.*

3. *Point-in-Time Recovery*

   Point-in-Time Recovery adalah salah satu metode *recovery* basis data dalam PostgreSQL. *Point-in-Time Recovery* merujuk pada pemulihan basis data berdasarkan perubahan yang terjadi hingga ke suatu waktu tertentu. Pemulihan ini biasanya dilakukan setelah DBMS melakukan pemulihan cadangan penuh, sehingga ketika pemulihan dilakukan, server sudah berada pada *state* ketika proses *backup* tersebut dilakukan. Kemudian, DBMS akan memulihkan server secara bertahap dari waktu *backup* dibuat hingga ke status server pada waktu terbaru. Untuk membantu hal ini, DBMS menyediakan suatu alur waktu atau *timeline* yang berisi rangkaian *log* yang tercipta dari proses *Write-Ahead Log*. DBMS akan menyimpan berbagai *timeline* yang telah dibuat dan *timeline* ini memiliki suatu ID yang unik. Hal ini berguna ketika pengguna ingin mengembalikan versi dari basis data namun versi dari basis data tersebut tidak

diketahui, sehingga pengguna dapat melakukan *recovery* dengan *timeline* ada dan tersimpan di dalam DBMS beberapa kali agar pengguna dapat menentukan versi yang dicari dari basis data. *Timeline* tersebut dibuat di dalam suatu file teks dan pengguna dapat menambahkan komentar pada file tersebut. Hal ini berguna ketika pengguna memiliki suatu kumpulan *timeline* dalam jumlah yang banyak.

Berdasarkan ketiga metode *recovery* tersebut, kami juga melakukan simulasi terhadap *recovery* pada DBMS. Hal ini kami lakukan dengan cara mematikan basis data saat suatu proses di dalam basis data masih berlangsung. Hal ini ditujukan agar pelayanan basis data terhenti saat tengah memproses dan hal ini dapat menimbulkan *error* pada basis data. Hal ini dapat dilakukan dengan cara menjalankan perintah berikut ini pada *Command Prompt*.

```
pg_ctl -D "C:\Program Files\PostgreSQL\13\data" stop
```

Perlu diperhatikan bahwa untuk menjalankan perintah tersebut, *Command Prompt* yang dijalankan harus dijalankan sebagai *Run as Administrator*, sehingga perintah tersebut dapat berjalan. Pada dasarnya, program tersebut memberhentikan proses dari DBMS, yang dalam hal ini adalah server dari basis data. Selain itu, **"C:\Program Files\PostgreSQL\13\data"** dapat diganti dengan alamat penyimpanan file dari basis data yang digunakan. Sementara itu, untuk menyalakan kembali basis data, kita dapat menggunakan perintah yang dijalankan seperti yang ada di bawah. Perlu diperhatikan bahwa perintah ini juga harus dijalankan sebagai Administrator (*Run as Administrator*).

```
pg_ctl -D "C:\Program Files\PostgreSQL\13\data" start
```

Dengan menggunakan perintah di atas, kita dapat menghentikan server PostgreSQL, meskipun server tengah menjalankan suatu perintah. Sesuai dengan hal tersebut, kami melakukan simulasi terhadap proses *recovery* basis data. Selain itu, untuk dapat melihat *log* pada basis data dengan jelas, kita juga perlu melakukan sedikit modifikasi pada *file* **postgresql.conf**. Perubahan yang dibuat pada dasarnya ditujukan agar *log* yang dicatatkan pada *file log* menjadi lebih jelas. Hal ini dilakukan dengan melakukan perubahan terhadap beberapa kriteria dalam *file* tersebut. Berikut ini adalah perubahan-perubahan yang dilakukan pada *file* **postgresql.conf**.

```
log_statement = 'all'
log_directory = 'pg_log'
log_filename = 'postgresql-%Y-%m-%d_%H%M%S.log'
logging_collector = on
```

```
log_min_error_statement = error
```

Pada simulasi ini, kami membuat suatu basis data baru dan mengisikannya dengan data yang baru pula. Pada basis data yang baru, hanya ada 1 tabel dan tabel tersebut hanya memiliki 5 buah data. Berikut ini adalah tampilan dari basis data yang akan digunakan.

```
test=# select * from test;
 id | data
----+------
  1 | A
  2 | B
  3 | C
  4 | D
  5 | E
(5 rows)
```

Untuk melakukan simulasi kegagalan, kita dapat menjalankan perintah seperti pada sebelumnya dijelaskan. Pada percobaan ini, kita akan menjalankan **pg_ctl -D "C:\Program Files\PostgreSQL\13\data" stop** Perintah tersebut dijalankan secara berurutan tanpa adanya perintah atau aksi lain di antaranya. Setelah itu, kita masukkan sembarang kueri ke dalam basis data tersebut. Dalam percobaan ini, kami menggunakan perintah **select pg_sleep_until('5 minutes');** agar basis data tetap bekerja. Perintah ini pada dasarnya meminta basis data untuk melakukan *sleep* selama 5 menit. Setelah itu, kita menjalankan perintah di atas. Hasil dari eksekusi perintah ini adalah sebagai berikut.

```
test=# select pg_sleep_for('5 minutes');
FATAL:  terminating connection due to administrator command
server closed the connection unexpectedly
        This probably means the server terminated abnormally
        before or while processing the request.
The connection to the server was lost. Attempting reset: Failed.
```

Setelah itu, kita jalankan kembali basis data dengan perintah **pg_ctl -D "C:\Program Files\PostgreSQL\13\data" start** dan kami masuk kembali ke dalam PostgreSQL sebagai pengguna. Pada dasarnya, data setelah terjadinya kegagalan pada basis data tidak mengalami perubahan. Berikut ini adalah hasil isi dari basis data setelah terjadinya kegagalan.

```
test=# select * from test;
 id | data
----+------
  1 | A
  2 | B
  3 | C
  4 | D
  5 | E
(5 rows)
```

Data dalam basis data setelah terjadinya kegagalan basis data tidak berubah. Hal ini dikarenakan kegagalan terjadi karena terputusnya koneksi antara klien dengan server basis data, sehingga tidak ada data yang hilang. Ketika kita mematikan sistem secara paksa ketika sistem sedang melakukan suatu proses, sistem akan menuliskan *log* baru terkait hal ini. Berikut ini adalah *log* ketika sistem dimatikan dengan perintah di atas.

```
[13892] LOG:  statement: select * from test;
[13892] LOG:  statement: insert into test values (6, 'F');
[13892] LOG:  statement: select * from test;
[13892] LOG:  statement: delete from test where id = 6;
[13892] LOG:  statement: select * from test;
[13892] LOG:  statement: select pg_sleep_for('5 minutes');
[17872] LOG:  received fast shutdown request
[17872] LOG:  aborting any active transactions
[13892] FATAL:  terminating connection due to administrator command
[13892] STATEMENT:  select pg_sleep_for('5 minutes');
[17872] LOG:  background worker "logical replication launcher" (PID 1588) exited with exit code 1
[16308] LOG:  shutting down
[13824] FATAL:  the database system is shutting down
[17872] LOG:  database system is shut down
```

Log untuk hasil eksekusi ini terdapat pada bagian /data/log pada *folder* tempat data terkait PostgreSQL disimpan. Pada log tersebut, DBMS akan menerima perintah dari administrator untuk segera mematikan server dan sehubungan dengan hal tersebut, DBMS akan membatalkan semua transaksi yang aktif, yang dalam hal ini adalah perintah **`select pg_sleep_until('5 minutes');`**. DBMS akan mematikan semua koneksi pengguna yang terhubung dengan server. DBMS akan menyimpan perintah yang dibatalkan secara paksa tersebut. DBMS juga akan menuliskan *error* terkait kegagalan yang terjadi. Dalam hal ini, sistem mengeluarkan *exit code* dan menuliskan PID yang menyatakan ID proses dimana basis data berjalan. Setelah itu, basis data akan melakukan *shut down* terhadap server basis data dan sistem basis data. Pada posisi pengguna, karena koneksi kepada basis data hilang, maka sistem akan berusaha untuk menyambungkan kembali pengguna dengan basis data. Namun karena dinyatakan bahwa basis data dimatikan, maka upaya sistem untuk menyambungkan kembali pengguna dan basis data gagal.

Pada tangkapan layar di atas, kita dapat melihat bahwa itu adalah satu log yang terdiri dari beberapa entri log. Entri log sendiri berisi perubahan yang dibuat atau dimasukkan ke dalam basis data pada suatu sesi transaksi. Dalam hal ini, satu sesi

menandakan masa pengguna *login* ke dalam basis data hingga akhir sesi. Suatu sesi berakhir ketika suatu basis data mengalami *error*. Ketika terjadi *error* pada basis data, maka penulisan *log* akan dilakukan pada *file* baru. Hal ini menyebabkan satu *file* penyimpan *log* akan menyimpan *log* dari awal (masa *login* pengguna) hingga terjadi *failure* pada basis data.

Ketika basis data mengalami kegagalan, DBMS akan melakukan *scan* terhadap *log* hingga ditemukan *checkpoint*. Basis data dari *checkpoint* tersebut akan diperbarui dengan *log* yang telah dibuat oleh basis data. Hal ini ditujukan agar versi dari basis data *checkpoint* mengikuti versi dari basis data yang baru sebelum terjadinya kegagalan. DBMS akan melakukan pembacaan terhadap *log* dan setiap entrinya serta melakukan eksekusi terhadap setiap kueri yang tercatat di dalam *log* tersebut sesuai dengan urutan. Hal ini dilakukan hingga versi dari basis data sudah sesuai.

BAB II
KESIMPULAN DAN SARAN

1. Kesimpulan

Dari tugas yang telah kami buat, kami melakukan analisis pada berbagai algoritma concurrency control, dan memperoleh kesimpulan pada kondisi operasi cukup sederhana dan contention cukup tinggi baik digunakan Simple Locking algorithm (yang kami uji dengan Exclusive-only Lock). Sedangkan pada kondisi konflik atau contention rendah dapat digunakan alternatif yaitu algoritma OCC, dan pada kondisi memerlukan derajat konkurensi tinggi dan transaksi yang kompleks dapat menggunakan alternatif MVCC, namun dengan catatan akan membutuhkan RAM lebih untuk menyimpan banyak versi.

Selain itu pada eksplorasi *recovery*, kami dapat menyimpulkan bahwa proses *recovery* dalam DBMS PostgreSQL melibatkan 3 metode, yaitu *Write-Ahead Log*, *Continuous Archiving*, dan *Point-in-Time Recovery*. *Write-Ahead Log* adalah proses *recovery* yang melibatkan pencatatan perubahan basis data ke dalam memori yang dilakukan jika perubahan sudah dicatat ke dalam memori. *Continuous Archiving* adalah proses *recovery* yang melibatkan proses *backup* pada tingkatan *file system* dan membuat *log* dari proses *Write-Ahead Logging* berdasarkan semua transaksi yang sudah dilakukan di dalam basis data. *Point-in-Time Recovery* adalah proses *recovery* yang melibatkan perubahan pada basis data hingga mencapai titik pada suatu waktu tertentu. Ketika percobaan proses *recovery* dijalankan, basis data akan melakukan pencatatan terhadap transaksi dalam *log*. Ketika basis data mengalami kegagalan, sistem akan melakukan *recovery* dengan menggunakan basis data *checkpoint* dan basis data tersebut diperbarui sesuai dengan *log* yang telah dibuat oleh DBMS hingga ke waktu tertentu, yaitu waktu sebelum terjadinya kegagalan.

2. Saran

Untuk mempermudah pengerjaan tugas ini kedepannya, berikut ini adalah beberapa hal yang kelompok kami sarankan.

- Terkait dengan spesifikasi tugas, kami menyarankan agar pada spek yang terkait dengan kode program, input dan output program terdefinisikan dengan jelas, akan lebih baik lagi jika ada contohnya.
- Pada concurrency control dapat membuat parametrik lain selain waktu eksekusi untuk ditampilkan, seperti puncak penggunaan memori, dengan tujuan dapat lebih memahami aspek memory usage tiap algoritma.

PEMBAGIAN KERJA DALAM KELOMPOK

NIM	Nama	Bagian yang Dikerjakan
13518102	Mohamad Falah Sutawindaya	Eskplorasi Recovery (membantu continuous archiving) dan membantu kesimpulan
13518109	Andjani Kiranadewi	Concurrency Control - Simple Locking Algorithm
13518140	Hansel Grady Daniel Thamrin	Eksplorasi *Recovery*
13518141	Muhammad Daffa Dinaya	Concurrency Control - OCC Algorithm
13518146	Naufal Prima Yoriko	Concurrency Control - MVCC Algorithm (Implementasi dan Testing), Analisis Algoritma pada Concurrency Control

REFERENSI

Abraham Silberschatz, Henry F. Korth, and Sudharshan .S. 2019. Database System Concepts 7* Edition. New York: McGraw-Hill.

MySQL. 2020. *1.5 Point-in-Time (Incremental) Recovery*. https://dev.mysql.com/doc/mysql-backup-excerpt/8.0/en/point-in-time-recovery.html. Diakses 4 November 2020.

PostgreSQL. 2020. *29.2. Write-Ahead Logging (WAL)*. https://www.postgresql.org/docs/9.1/wal-intro.html. Diakses 3 November 2020.

PostgreSQL. 2020. *24.3. Continuous Archiving and Point-in-Time Recovery (PITR)*. https://www.postgresql.org/docs/9.1/continuous-archiving.html. Diakses 4 November 2020.

Slide Kuliah *Concurrency Control*. 2020. Bandung: Institut Teknologi Bandung.

Slide Kuliah *Recovery System*. 2020. Bandung: Institut Teknologi Bandung.

Spesifikasi Tugas Besar https://bit.ly/Tubes2MBD2020

https://stackoverflow.com/questions/5751271/optimistic-vs-multi-version-concurrency-control-differences#:~:text=MVCC%20is%20an%20optimistic%20technique,transactions%20which%20span%20multiple%20objects.&text=To%20directly%20reply%20to%20the,optimistic%20concurrency%20control%20(OCC). Diakses 13 November 2020.

https://www.cyberciti.biz/faq/debian-rhel-centos-redhat-suse-hotplug-cpu/. Diakses 11 November 2020